

SEAL - Configuración Segura

Hugo Mauricio Romero Rodríguez
Miguel Angel Leal Perez
Miguel Angel Durazno Martinez
Omar Hernandez Cervantes

DS01SV-25

Hector Saldaña Benitez

Desarrollo Web Integral

1 de Julio de 2026
San Juan del Río, Qro.

Bloque 1: Arquitectura de la Información y Gestión de Secretos

Este bloque evalúa que la infraestructura de código base y la información sensible estén completamente aisladas.

1.1 Aislamiento de Credenciales (.env)

- Aislamiento de Credenciales (.env): Todas las llaves API, tokens de autenticación, contraseñas y cadenas de conexión a bases de datos están parametrizadas en archivos de variables de entorno.

Estado en SEAL: Cumple.

Evidencia:

El backend carga configuraciones sensibles desde variables de entorno en ``Back/src/config/env.js``. Entre ellas:

- `JWT_SECRET`
- `FIREBASE_SERVICE_ACCOUNT`
- `FIREBASE_STORAGE_BUCKET`
- `TOKEN_PEPPER`
- `SMTP_USER`
- `SMTP_PASS`
- `GEMINI_API_KEY`
- `ADMIN_BOOTSTRAP_EMAIL`
- `ADMIN_BOOTSTRAP_PASSWORD`

El frontend también usa configuración por entorno mediante:

- `NEXT_PUBLIC_API_URL`

Archivos relacionados:

- `Back/.env.example`
- `Front/.env.local.example`
- `Back/src/config/env.js`

Conclusion:

El proyecto no deja credenciales reales escritas directamente en el código fuente. Los secretos se configuran mediante archivos ``.env`` y variables de entorno.

Recomendacion:

Mantener los ``.env`` reales fuera del repositorio y usar un gestor de secretos en producción.

1.2 Protección del Repositorio (.gitignore)

- Protección del Repositorio (.gitignore): El archivo `.gitignore` esta correctamente configurado y se verifico que no se suban por error archivos `.env`, `node_modules`, carpetas de compilación ni secretos.

Estado en SEAL: Cumple.

Evidencia:

Existe un archivo `.gitignore` en la raíz del proyecto. Este ignora:

- `node_modules/`
- `.next/`
- `dist/`
- `build/`
- `coverage/`
- `.env`
- `.env.*`
- `Back/secrets/`
- archivos `.pem`, `.key`, `.p12`, `.pfx`
- archivos de credenciales de Firebase
- `Back/.local-storage/`
- archivos de logs

La revision del estado de Git muestra que los siguientes elementos estan ignorados correctamente:

- `Back/.env`
- `Front/.env.local`
- `Front/.next/`
- `Back/secrets/`
- `Back/.local-storage/`

Conclusion:

El repositorio esta protegido contra la inclusion accidental de dependencias, secretos, credenciales, builds y archivos generados localmente.

Recomendacion:

Antes de entregar o subir a GitHub, ejecutar una revision final con:

```
git status --ignored
```

1.3 Entornos Separados

- Entornos Separados: El código distingue explícitamente entre variables para desarrollo local y producción.

Estado en SEAL: Cumple parcialmente.

Evidencia encontrada:

El backend utiliza:

- `NODE_ENV`
- `PORT`
- `FRONT_RESET_URL`
- `FRONT_SIGN_URL`
- `FIREBASE_STORAGE_BUCKET`
- `ENABLE_LOCAL_DRAFT_CLEANUP`

El frontend utiliza:

- `NEXT_PUBLIC_API_URL`

Además, existen archivos de ejemplo:

- `Back/.env.example`
- `Front/.env.local.example`

Limitación:

No hay archivos separados como:

- `.env.development`
- `.env.production`

Conclusion:

El proyecto soporta variables de entorno y distingue configuraciones mediante ``NODE_ENV``, pero todavía no separa formalmente archivos de desarrollo y producción.

Recomendación:

Crear plantillas separadas:

- `Back/.env.development.example`
- `Back/.env.production.example`
- `Front/.env.development.example`
- `Front/.env.production.example`

Bloque 2: Validación, Saneamiento y Manejo de Entradas de Usuario

Este bloque verifica las defensas del código contra ataques comunes de inyección, como SQL Injection, XSS y Command Injection.

2.1 Validación Estricta de Tipos en Backend

- Validación Estricta de Tipos (Backend): Todas las solicitudes de la API se validan con esquemas de tipado fuerte o librerías de validación antes de ejecutar lógica de negocio.

Estado en SEAL: Cumple.

Evidencia:

El backend utiliza `Zod` para validar datos de entrada. Existen esquemas en:

- `Back/src/validators/auth.schemas.js`
- `Back/src/validators/users.schemas.js`
- `Back/src/validators/contracts.schemas.js`
- `Back/src/validators/assignments.schemas.js`
- `Back/src/validators/aiChats.schemas.js`

También hay validaciones específicas dentro de controladores, por ejemplo:

- `Back/src/controllers/clientProfile.controller.js`
- `Back/src/controllers/mobileSignature.controller.js`

Ejemplos de validación:

- El login valida email y password.
- Los usuarios solo pueden tener rol `ADMIN` o `CLIENT`.
- Las versiones de contrato deben ser enteros positivos.
- Los mensajes tienen longitud máxima.
- Los estados de chat solo pueden ser `OPEN` o `CLOSED`.
- La firma debe llegar en base64 y corresponder a una imagen PNG.
- RFC, CURP y teléfono se validan con expresiones regulares.

Conclusion:

El proyecto aplica validación sistemática antes de ejecutar la lógica principal del backend.

2.2 Saneamiento de Entradas

- Saneamiento de Entradas (Sanitization): Las entradas de texto libre que se renderizan en frontend o se almacenan en base de datos se limpian para remover HTML o scripts maliciosos.

Estado en SEAL: Cumple parcialmente.

Evidencia:

El proyecto cuenta con una función ``escapeHtml`` en ``Back/src/utls/html.js``. Esta función convierte caracteres peligrosos:

- `&`
- `<`
- `>`
- `"`
- `'`

También se utiliza en ``Back/src/utls/placeholderRender.js`` para escapar los valores reales insertados en las plantillas de contrato.

Esto protege los datos provenientes de usuarios, por ejemplo:

- nombre del empleado
- correo
- RFC
- CURP
- telefono
- dirección
- valores de empresa insertados como placeholders

Riesgo identificado:

El frontend renderiza HTML de contratos mediante ``dangerouslySetInnerHTML`` en pantallas como:

- `Front/src/app/cliente/dashboard/page.tsx`
- `Front/src/app/(dashboard)/contratos/page.tsx`
- `Front/src/app/(dashboard)/plantillas/ia/page.tsx`

Aunque los valores de placeholders se escapan, las plantillas completas pueden venir de:

- archivos ``.docx``
- edición manual del administrador
- generación con IA

No se encontró una sanitización HTML formal con una librería como ``DOMPurify`` o ``sanitize-html``.

Conclusion:

SEAL protege los valores insertados en placeholders, pero debe reforzar el saneamiento del HTML completo que se renderiza en el frontend.

Recomendacion:

Agregar una política de sanitizacion HTML antes de guardar o renderizar plantillas. Por ejemplo:

- Backend: `sanitize-html`
- Frontend: `DOMPurify`

Tambien se recomienda bloquear etiquetas y atributos peligrosos como:

- `script`
- `iframe`
- `object`
- `embed`
- `onerror`
- `onclick`
- `javascript:`

2.3 Consultas Parametrizadas / ORM / ODM

- Consultas Parametrizadas (ORMs): No existen concatenaciones directas de variables de texto dentro de consultas de bases de datos.

Estado en SEAL: Cumple.

Evidencia:

SEAL no utiliza SQL ni construye consultas mediante concatenación de strings. El backend usa Firebase Admin SDK y Firestore mediante métodos seguros como:

- `collection()`
- `doc()`
- `where()`
- `get()`
- `set()`
- `update()`
- `runTransaction()`

Ejemplos de servicios que usan Firestore:

- `Back/src/services/users.service.js`
- `Back/src/services/contracts.service.js`
- `Back/src/services/assignments.service.js`

- `Back/src/services/contractVersions.service.js`
- `Back/src/services/aiChats.service.js`

Conclusion:

No se observaron consultas SQL ni concatenaciones directas que expongan a SQL Injection. El uso del SDK de Firestore reduce ese riesgo.

Recomendacion:

Mantener todas las operaciones de base de datos a traves del SDK o servicios internos, evitando consultas construidas manualmente.

2.4 Validación en Carga de Archivos

- Validación en Carga de Archivos: El sistema valida extension, tipo MIME real o estructura del archivo y limita el tamaño máximo.

Estado en SEAL: Cumple.

Evidencia:

La subida de contratos se maneja con `multer` en:

- `Back/src/controllers/adminContracts.controller.js`

Controles implementados:

- `multer.memoryStorage()`
- limite de tamaño mediante `MAX\_DOCX\_MB`
- rechazo de archivos que superan el limite con `LIMIT\_FILE\_SIZE`

La validación del archivo `.docx` se realiza en:

- `Back/src/services/docxValidation.service.js`

Controles implementados:

- Valida extension `.docx`.
- Rechaza archivos `.docm`.
- Valida MIME esperado de Word.
- Verifica encabezado ZIP `PK`.
- Verifica que exista `word/document.xml`.
- Rechaza macros si existe `word/vbaProject.bin`.

Conclusion:

La carga de archivos esta protegida con validaciones de tamaño, extension, MIME, estructura interna y bloqueo de macros.

Recomendacion:

Agregar pruebas automatizadas con archivos:

- `.docx` valido
- `.docm`
- archivo renombrado a `.docx` sin estructura ZIP
- `.docx` con macros
- archivo que exceda `MAX_DOCX_MB`

Bloque 3: Autenticación, Control de Accesos y Gestión de Sesiones

Este bloque garantiza que la identidad de los usuarios este protegida y que no existan vulnerabilidades de escalacion de privilegios.

3.1 Cifrado de Contraseñas

- Cifrado de Contraseñas: Las contraseñas en la base de datos se almacenan con hashing seguro. Nunca se almacenan en texto plano.

Estado en SEAL: Cumple.

Evidencia:

El backend usa `'bcryptjs'` en:

- `Back/src/utils/password.js`

La función `'hashPassword'` genera un salt con costo 12:

```
const salt = await bcrypt.genSalt(12);
return bcrypt.hash(plain, salt);
```

La verificación se realiza con:

```
return bcrypt.compare(plain, hash);
```

Este mecanismo se usa en:

- creación de usuarios
- bootstrap de administrador
- cambio de contraseña
- recuperación de contraseña
- login

Conclusion:

Las contraseñas no se guardan en texto plano. Se almacenan como hash con `bcrypt` y salt.

3.2 Principio de Menor Privilegio (RBAC)

- Principio de Menor Privilegio (RBAC): El sistema implementa control de acceso basado en roles y valida permisos en cada endpoint del backend.

Estado en SEAL: Cumple.

Evidencia:

El backend implementa:

- `Back/src/middlewares/authJWT.js`
- `Back/src/middlewares/requireRole.js`

Las rutas administrativas usan:

```
router.use(authJWT, requireRole("ADMIN"));
```

Las rutas del cliente usan:

```
router.use(authJWT, requireRole("CLIENT"));
```

Esto se aplica en:

- `Back/src/routes/admin.routes.js`
- `Back/src/routes/adminContracts.routes.js`
- `Back/src/routes/adminAssignments.routes.js`
- `Back/src/routes/adminAiChats.routes.js`
- `Back/src/routes/clientAssignments.routes.js`

Ademas, en los servicios y controladores se valida que un cliente solo pueda consultar o firmar sus propias asignaciones.

Conclusion:

El control de accesos no depende solo del frontend. La API valida autenticación y rol en backend.

Recomendacion:

Para una version productiva, dividir el rol `ADMIN` en permisos mas granulares:

- administrador general
- area legal
- recursos humanos
- soporte

3.3 Seguridad de Tokens JWT

- Seguridad de Tokens (JWT): Los JWT usan secret, expiran y no contienen información altamente confidencial.

Estado en SEAL: Cumple.

Evidencia:

El token se firma en:

- `Back/src/utils/token.js`

Con:

```
jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
```

La expiración se configura en:

- `Back/src/config/env.js`

Valor por defecto:

```
JWT_EXPIRES_IN=1h
```

El payload del token contiene:

- `userId`
- `role`

No contiene:

- `password`
- `hash de password`
- `RFC`
- `CURP`
- `dirección`
- `datos bancarios`
- `firma digital`

Conclusion:

El JWT contiene solo datos mínimos de identidad y autorización, tiene expiración y usa un secret configurado por entorno.

Recomendacion:

En producción:

- Usar un ``JWT_SECRET`` largo y aleatorio.
- Rotar secretos periódicamente.
- Evitar valores de ejemplo como ``change-me``.
- Evaluar refresh tokens con rotación.

3.4 Atributos de Cookies Seguras

- Atributos de Cookies Seguras: Las cookies de sesión cuentan con `HttpOnly`, `Secure` y `SameSite`.

Estado en SEAL: No cumple en el estado actual.

Evidencia encontrada:

El frontend almacena la sesión en `localStorage`, no en cookies:

- `Front/src/lib/auth.ts`

Se usan claves:

- `seal_token`
- `seal_user`

El token se lee desde `localStorage` y se manda en el header:

```
Authorization: Bearer <token>
```

Conclusion:

No existen cookies de sesión, por lo tanto no se pueden aplicar banderas `HttpOnly`, `Secure` ni `SameSite`.

Esto no rompe el funcionamiento del sistema, pero desde seguridad web moderna es menos seguro que cookies `HttpOnly`, ya que un XSS podría leer el token desde `localStorage`.

Recomendacion:

Migrar el manejo de sesión a cookies seguras:

- `HttpOnly`
- `Secure`
- `SameSite=Lax` o `SameSite=Strict`
- expiración corta
- refresh token separado

Bloque 4: Encabezados de Seguridad HTTP y Configuración de Red

Este bloque verifica que el servidor y las respuestas HTTP mitiguen vectores de ataque desde el navegador.

4.1 Manejo Correcto de CORS

- [~] Manejo Correcto de CORS: La política CORS esta configurada explícitamente permitiendo solo el dominio del frontend en producción.

Estado en SEAL: Cumple parcialmente.

Evidencia:

El backend activa CORS en:

- `Back/src/app.js`

Con:

```
app.use(cors());
```

Limitacion:

La configuración actual no restringe explícitamente orígenes. Al usar `cors()` sin opciones, el backend queda abierto a solicitudes desde distintos orígenes.

Esto puede ser aceptable en desarrollo local, pero no es recomendable para producción.

Conclusion:

SEAL usa CORS, pero todavía no aplica una política restrictiva por entorno.

Recomendacion:

Configurar CORS con una variable como:

```
FRONTEND_ORIGIN=https://seal.midominio.com
```

Y aplicar:

```
app.use(cors({
  origin: process.env.FRONTEND_ORIGIN,
  credentials: true
}));
```

Para desarrollo:

```
FRONTEND_ORIGIN=http://localhost:3000
```

4.2 Protección Helmet / Encabezados HTTP

- Protección Helms/Encabezados: El servidor implementa middleware de seguridad como `helmet`.

Estado en SEAL: Cumple.

Evidencia:

El backend importa y utiliza `helmet` en:

- `Back/src/app.js`

Con:

```
const helmet = require("helmet");
app.use(helmet());
```

Helmet ayuda a configurar cabeceras de seguridad como:

- `X-Content-Type-Options`
- `X-Frame-Options`

- protecciones basicas contra clickjacking
- políticas iniciales relacionadas con seguridad del navegador

Conclusion:

El backend implementa una capa de protección de encabezados HTTP mediante `helmet`.

Recomendacion:

Para producción, revisar y personalizar la política CSP según los recursos reales que use el frontend.

Bloque 5: Manejo de Errores, Bitácoras y Logs

Este bloque previene que el sistema revele pistas que faciliten el mapeo de vulnerabilidades a atacantes externos.

5.1 Abstracción de Errores en Producción

- Abstracción de Errores en Producción: Las excepciones del backend no envían stack traces ni detalles internos al cliente.

Estado en SEAL: Cumple.

Evidencia:

El backend cuenta con middleware global de errores:

- `Back/src/middlewares/errorHandler.js`

Este responde:

```
{
  "ok": false,
  "message": "Internal server error"
}
```

Tambien se usa un patron de respuestas controladas con `ok` y `fail`, evitando enviar trazas completas al usuario final.

Ejemplos de errores controlados:

- 401: token faltante o invalido
- 403: acceso prohibido
- 400: body invalido
- 404: recurso no encontrado
- 409: conflicto de estado

Conclusion:

El sistema evita exponer stack traces al cliente y responde con mensajes controlados.

Observacion:

Algunos errores de validación incluyen `details`. Esto es útil para formularios, pero debe revisarse en producción para asegurar que no se filtren datos internos.

5.2 Logs de Seguridad Centralizados

- [~] Logs de Seguridad Centralizados: Los errores detallados se registran internamente para análisis de desarrolladores.

Estado en SEAL: Cumple parcialmente.

Evidencia encontrada:

El backend usa:

- `morgan( "dev" )` en `Back/src/app.js``
- `console.error` en `Back/src/middlewares/errorHandler.js``

Esto permite registrar errores y peticiones en la consola del servidor durante desarrollo.

Limitacion:

No se encontró un sistema formal de logs centralizados, archivos protegidos, rotación de logs o separación de niveles por entorno.

No se observó integración con herramientas como:

- Winston
- Pino
- Sentry
- Cloud Logging
- Datadog

Conclusion:

El sistema registra información útil para desarrollo, pero no cuenta aún con una estrategia robusta de logging para producción.

Recomendacion:

Agregar logging estructurado y seguro:

- niveles `info`, `warn`, `error`
- logs sanitizados
- omitir tokens, contraseñas, firmas base64 y datos personales
- agregar IDs de correlación

- almacenar logs en un sistema restringido

Conclusion Final

El proyecto SEAL presenta una base solida de codificación segura para su etapa actual. Los controles mas fuertes se encuentran en:

- gestión de secretos mediante variables de entorno
- protección del repositorio con `.gitignore`
- validación de entradas con Zod
- control de acceso por roles
- autenticación con JWT
- hashing de contraseñas con bcrypt
- validación de archivos `.docx`
- uso de Helmet
- manejo controlado de errores

Los principales puntos pendientes para fortalecer el proyecto son:

- separar formalmente entornos de desarrollo y producción
- sanitizar HTML completo antes de renderizar plantillas
- migrar tokens de `localStorage` a cookies seguras
- restringir CORS por dominio
- implementar logs centralizados y sanitizados

En conclusion, SEAL cumple la mayoría de los principios evaluados por el checklist. El sistema no esta terminado al 100%, pero ya incorpora practicas importantes de seguridad desde el backend y tiene una arquitectura clara para seguir endureciendose antes de un despliegue productivo.

Anexo de Código: Evidencia de Codificación Segura en SEAL

Este anexo complementa el documento original. Para cada punto del checklist se agrega código real del proyecto y una nota breve sobre cómo demostrarlo en la app, terminal o repositorio.

#	Control	Estado	Demostración recomendada
1.1	Aislamiento de Credenciales (.env)	Cumple	Mostrar Back/src/config/env.js. No abrir valores reales de Back/.env durante la exposición.
1.2	Protección del Repositorio (.gitignore)	Cumple	Mostrar .gitignore y ejecutar git status --ignored para ver .env, .next, secrets y storage local ignorados.
1.3	Entornos Separados	Cumple parcialmente	Mostrar que existe NODE_ENV y archivos de ejemplo. Explicar que faltan plantillas separadas de producción/desarrollo.
2.1	Validación Estricta de Tipos (Backend)	Cumple	En la app, intentar login con correo inválido. En código, mostrar zod y safeParse.
2.2	Sanitización de Entradas (Sanitization)	Cumple parcialmente	Mostrar escapeHtml como control positivo y dangerouslySetInnerHTML como pendiente de sanitización completa.
2.3	Consultas Parametrizadas / ORM	Cumple	Mostrar servicios Firestore. No hay SQL concatenado; se usa SDK con collection, doc, where y transacciones.
2.4	Validación en Carga de Archivos	Cumple	En Admin > Plantillas intentar subir un archivo que no sea .docx. Mostrar validación de extensión, MIME, ZIP y macros.
3.1	Cifrado de Contraseñas	Cumple	Mostrar bcrypt.genSalt(12), bcrypt.hash y verifyPassword. No mostrar contraseñas reales.
3.2	Principio de Menor Privilegio (RBAC)	Cumple	Entrar como admin y cliente. En código mostrar authJWT, requireRole y rutas ADMIN/CLIENT.
3.3	Seguridad de Tokens (JWT)	Cumple	Hacer login y mostrar DevTools > Local Storage > seal_token. En código mostrar payload mínimo y expiración.
3.4	Atributos de Cookies Seguras	No cumple	Mostrar en DevTools que la sesión vive en Local Storage, no en cookies HttpOnly/Secure/SameSite.
4.1	Manejo Correcto de CORS	Cumple parcialmente	Mostrar Back/src/app.js: usa cors(), pero sin origen restringido. Explicar que en producción debe limitarse.
4.2	Helmet / Encabezados HTTP	Cumple	Con backend prendido ejecutar curl -I http://localhost:3001/health y mostrar headers de seguridad.
5.1	Abstracción de Errores en Producción	Cumple	Ejecutar curl -i http://localhost:3001/admin/users sin token. Debe devolver 401 sin stack trace.
5.2	Logs de Seguridad Centralizados	Cumple parcialmente	Dejar visible la terminal del backend: Morgan registra requests. Explicar que falta logging centralizado tipo Winston/Pino/Sentry.

1.1 - Aislamiento de Credenciales (.env)

Estado: Cumple

Como demostrarlo: Mostrar Back/src/config/env.js. No abrir valores reales de Back/.env durante la exposicion.

Archivo: Back/src/config/env.js (lineas 23-42)

```
23 | const PORT = getInt("PORT", 3001);
24 | const NODE_ENV = process.env.NODE_ENV || "development";
25 |
26 | const JWT_SECRET = mustGet("JWT_SECRET");
27 | const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || "1h";
28 |
29 | const FIREBASE_SERVICE_ACCOUNT = mustGet("FIREBASE_SERVICE_ACCOUNT");
30 | const FIREBASE_STORAGE_BUCKET = process.env.FIREBASE_STORAGE_BUCKET || "";
31 |
32 | const FRONT_RESET_URL = process.env.FRONT_RESET_URL ||
  | "http://localhost:3000/reset-password";
33 | const FRONT_SIGN_URL = process.env.FRONT_SIGN_URL || "http://localhost:3000/firma-movil";
34 | const PASSWORD_TOKEN_EXPIRES_MIN = getInt("PASSWORD_TOKEN_EXPIRES_MIN", 30);
35 | const TOKEN_PEPPER = mustGet("TOKEN_PEPPER");
36 | const RETURN_RESET_TOKEN_IN_RESPONSE = getBool("RETURN_RESET_TOKEN_IN_RESPONSE", false);
37 |
38 | const SMTP_HOST = process.env.SMTP_HOST || "smtp.gmail.com";
39 | const SMTP_PORT = getInt("SMTP_PORT", 587);
40 | const SMTP_USER = mustGet("SMTP_USER");
41 | const SMTP_PASS = mustGet("SMTP_PASS");
42 | const EMAIL_FROM = process.env.EMAIL_FROM || SMTP_USER;
```

Archivo: Back/.env.example (lineas 1-18)

```
1 | PORT=3001
2 | NODE_ENV=development
3 |
4 | JWT_SECRET=change-me
5 | JWT_EXPIRES_IN=1h
6 |
7 | FIREBASE_SERVICE_ACCOUNT=secrets/firebase-admin.json
8 | FIREBASE_STORAGE_BUCKET=
9 |
10 | FRONT_RESET_URL=http://localhost:3000/reset-password
11 | FRONT_SIGN_URL=http://localhost:3000/firma-movil
12 | PASSWORD_TOKEN_EXPIRES_MIN=30
13 | TOKEN_PEPPER=change-me
14 | RETURN_RESET_TOKEN_IN_RESPONSE=false
15 | INTERNAL_CRON_SECRET=
16 |
17 | ADMIN_BOOTSTRAP_EMAIL=admin@example.com
18 | ADMIN_BOOTSTRAP_PASSWORD=change-me-strong-password
```

1.2 - Proteccion del Repositorio (.gitignore)

Estado: Cumple

Como demostrarlo: Mostrar .gitignore y ejecutar git status --ignored para ver .env, .next, secrets y storage local ignorados.

Archivo: .gitignore (lineas 15-35)

```
15 | # Environment variables and local secrets
16 | .env
17 | .env.*
18 | **/.env
19 | **/.env.*
20 | !.env.example
21 | !.env.local.example
22 |
23 | # Firebase/admin credentials and other private keys
24 | Back/secrets/
25 | **/secrets/
26 | *.pem
27 | *.key
28 | *.p12
29 | *.pfx
30 | *service-account*.json
31 | *firebase-admin*.json
32 |
33 | # Local generated files and uploads
34 | Back/.local-storage/
```

```
35 | **/.local-storage/
```

Archivo: .gitignore (lineas 40-51)

```
40 | # Logs and debug files
41 | *.log
42 | npm-debug.log*
43 | yarn-debug.log*
44 | yarn-error.log*
45 | pnpm-debug.log*
46 |
47 | # OS/editor files
48 | .DS_Store
49 | Thumbs.db
50 | .vscode/
51 | .idea/
```

1.3 - Entornos Separados

Estado: Cumple parcialmente

Como demostrarlo: Mostrar que existe NODE_ENV y archivos de ejemplo. Explicar que faltan plantillas separadas de produccion/desarrollo.

Archivo: Back/src/config/env.js (lineas 23-36)

```
23 | const PORT = getInt("PORT", 3001);
24 | const NODE_ENV = process.env.NODE_ENV || "development";
25 |
26 | const JWT_SECRET = mustGet("JWT_SECRET");
27 | const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || "1h";
28 |
29 | const FIREBASE_SERVICE_ACCOUNT = mustGet("FIREBASE_SERVICE_ACCOUNT");
30 | const FIREBASE_STORAGE_BUCKET = process.env.FIREBASE_STORAGE_BUCKET || "";
31 |
32 | const FRONT_RESET_URL = process.env.FRONT_RESET_URL ||
  | "http://localhost:3000/reset-password";
33 | const FRONT_SIGN_URL = process.env.FRONT_SIGN_URL || "http://localhost:3000/firma-movil";
34 | const PASSWORD_TOKEN_EXPIRES_MIN = getInt("PASSWORD_TOKEN_EXPIRES_MIN", 30);
35 | const TOKEN_PEPPER = mustGet("TOKEN_PEPPER");
36 | const RETURN_RESET_TOKEN_IN_RESPONSE = getBool("RETURN_RESET_TOKEN_IN_RESPONSE", false);
```

Archivo: Front/.env.local.example (lineas 1-5)

```
1 | NEXT_PUBLIC_API_URL=http://localhost:3001
```

2.1 - Validacion Estricta de Tipos (Backend)

Estado: Cumple

Como demostrarlo: En la app, intentar login con correo invalido. En codigo, mostrar Zod y safeParse.

Archivo: Back/src/validators/auth.schemas.js (lineas 1-23)

```
1 | const { z } = require("zod");
2 |
3 | const loginSchema = z.object({
4 |   email: z.string().email(),
5 |   password: z.string().min(6),
6 | });
7 |
8 | const changePasswordSchema = z.object({
9 |   currentPassword: z.string().min(6),
10 |   newPassword: z.string().min(8),
11 | });
12 |
13 | // ===== Fase 2: Password reset por token =====
14 | const requestResetSchema = z.object({
15 |   email: z.string().email(),
16 | });
17 |
18 | const confirmResetSchema = z.object({
19 |   token: z.string().min(10),
20 |   newPassword: z.string().min(8),
21 | });
22 |
23 | module.exports = { loginSchema, changePasswordSchema, requestResetSchema,
  | confirmResetSchema };
```

Archivo: Back/src/validators/assignments.schemas.js (lineas 5-43)

```
5 | const PrecheckSchema = z.object({
6 |   client_id: Id,
7 |   contract_id: Id,
8 |   contract_version: z.number().int().min(1),
9 | });
10 |
11 | const CreateAssignmentSchema = z.object({
12 |   client_id: Id,
13 |   contract_id: Id,
14 |   contract_version: z.number().int().min(1),
15 |   initial_message: z.string().max(2000).optional(),
16 |   // company values se mandan como { "company.title": "...", ... }
17 |   company_values: z.record(z.string(), z.any()).default({}),
18 | });
19 |
20 | const SendMessageSchema = z.object({
21 |   text: z.string().min(1).max(5000),
22 | });
23 |
24 | const AiAskSchema = z.object({
25 |   question: z.string().min(1).max(2000),
26 | });
27 |
28 | const SetChatStatusSchema = z.object({
29 |   status: z.enum(["OPEN", "CLOSED"]),
30 | });
31 |
32 | const SignSchema = z.object({
33 |   signature_png_base64: z.string().min(20), // base64 o data URL
34 |   signature_bbox: z
35 |     .object({
36 |       x: z.number().optional(),
37 |       y: z.number().optional(),
38 |       width: z.number().optional(),
39 |       height: z.number().optional(),
40 |       page: z.number().optional(),
41 |     })
42 |     .optional(),
43 | });
```

Archivo: Back/src/controllers/auth.controller.js (lineas 7-18)

```
7 | async function login(req, res) {
8 |   const parsed = loginSchema.safeParse(req.body);
9 |   if (!parsed.success) return fail(res, "Invalid body", 400, parsed.error.flatten());
10 |
11 |   const { email, password } = parsed.data;
12 |   const user = await usersService.getByEmail(email);
13 |   if (!user) return fail(res, "Invalid credentials", 401);
14 |
15 |   if (user.status !== "ACTIVE") return fail(res, "User disabled", 403);
16 |
17 |   const match = await verifyPassword(password, user.password_hash);
18 |   if (!match) return fail(res, "Invalid credentials", 401);
```

2.2 - Saneamiento de Entradas (Sanitization)

Estado: Cumple parcialmente

Como demostrarlo: Mostrar escapeHtml como control positivo y dangerouslySetInnerHTML como pendiente de sanitización completa.

Archivo: Back/src/utills/html.js (lineas 3-18)

```
3 | function escapeHtml(value) {
4 |   if (value === null || value === undefined) return "";
5 |   return String(value)
6 |     .replace(/&/g, "&amp;")
7 |     .replace(/</g, "&lt;")
8 |     .replace(/>/g, "&gt;")
9 |     .replace(/"/g, "&quot;")
10 |    .replace(/'/g, "&#39;");
11 | }
12 |
13 | function sha256Hex(input) {
14 |   const buf = Buffer.isBuffer(input) ? input : Buffer.from(String(input), "utf8");
15 |   return crypto.createHash("sha256").update(buf).digest("hex");
16 | }
17 |
18 | module.exports = { escapeHtml, sha256Hex };
```

Archivo: Back/src/utills/placeholderRender.js (lineas 1-35)

```
1 | const { escapeHtml } = require("./html");
2 |
3 | /**
4 |  * Reemplaza:
5 |  * 1) Tokens: <span data-ph="employee.rfc">...</span>
6 |  * 2) Legacy: {{ employee.rfc }}
7 |  *
8 |  * Nota: Escapa HTML para evitar inyección.
9 |  */
10 | function renderTemplateHtml(templateHtml, values) {
11 |     let html = String(templateHtml || "");
12 |     const map = values || {};
13 |
14 |     // 1) Tokens: reemplaza el span completo por el valor
15 |     // Acepta cualquier etiqueta que tenga data-ph="key" (span normalmente).
16 |     html = html.replace(
17 |
18 |         /<([a-zA-Z0-9]+)([^\s]*)data-ph\s*=\s*"([a-zA-Z0-9_.]+)"([^\s]*)>([\s\S]*)<\/\1>/g,
19 |         (full, tag, pre, key) => {
20 |             const v = map[key];
21 |             if (v === undefined || v === null) return "";
22 |             return escapeHtml(v);
23 |         }
24 |     );
25 |
26 |     // 2) Legacy: {{ key }}
27 |     html = html.replace(/{{\s*([a-zA-Z0-9_.]+\s*)}}/g, (full, key) => {
28 |         const v = map[key];
29 |         if (v === undefined || v === null) return "";
30 |         return escapeHtml(v);
31 |     });
32 |     return html;
33 | }
34 |
35 | module.exports = { renderTemplateHtml };
```

Archivo: Front/src/app/(dashboard)/contratos/page.tsx (lineas 748-758)

```
748 |         </button>
749 |     </form>
750 | </div>
751 | ) : activeTab === "documento" ? (
752 |     <div className="note document-viewer">
753 |         {selected.resolved_html_snapshot ? (
754 |             <div
755 |                 className="document-page"
756 |                 dangerouslySetInnerHTML={{ __html: selected.resolved_html_snapshot }}
757 |             />
758 |         ) : (
```

Archivo: Front/src/app/cliente/dashboard/page.tsx (lineas 728-734)

```
728 |         ) : activeTab === "documento" ? (
729 |             <div className="note document-viewer client-document-viewer">
730 |                 {selected.resolved_html_snapshot ? (
731 |                     <div className={`document-page ${signatureImageUrl ?
732 | "document-page--signed" : ""}`}>
733 |                         <div dangerouslySetInnerHTML={{ __html:
734 | selected.resolved_html_snapshot }} />
735 |
736 |                         {(signatureImageUrl || signatureImageFailed) && (
```

2.3 - Consultas Parametrizadas / ORM

Estado: Cumple

Como demostrarlo: Mostrar servicios Firestore. No hay SQL concatenado; se usa SDK con collection, doc, where y transacciones.

Archivo: Back/src/services/users.service.js (lineas 4-21)

```
4 | const col = () => db().collection("users");
5 |
6 | async function getByEmail(email) {
7 |     const snap = await col().where("email", "=", email).limit(1).get();
8 |     if (snap.empty) return null;
9 |     const doc = snap.docs[0];
10 |     return { id: doc.id, ...doc.data() };
11 | }
12 |
```

```

13 | async function getById(id) {
14 |     const doc = await col().doc(id).get();
15 |     if (!doc.exists) return null;
16 |     return { id: doc.id, ...doc.data() };
17 | }
18 |
19 | async function list() {
20 |     const snap = await col().orderBy("created_at", "desc").get();
21 |     return snap.docs.map(d => ({ id: d.id, ...d.data() }));

```

Archivo: Back/src/services/assignments.service.js (lineas 317-337)

```

317 | async function listAssignmentsForAdmin({ status, chat_status, limit = 50 }) {
318 |     const q = col();
319 |     let query = q;
320 |     if (status) query = query.where("status", "=", status);
321 |     if (chat_status) query = query.where("chat_status", "=", chat_status);
322 |     const snap = await query.orderBy("created_at", "desc").limit(limit).get();
323 |     return snap.docs.map((d) => d.data());
324 | }
325 |
326 | async function listAssignmentsForClient({ client_id, status, limit = 50 }) {
327 |     let query = col().where("client_id", "=", client_id);
328 |     if (status) query = query.where("status", "=", status);
329 |     const snap = await query.orderBy("created_at", "desc").limit(limit).get();
330 |     return snap.docs.map((d) => d.data());
331 | }
332 |
333 | async function getAssignmentById(id) {
334 |     const doc = await col().doc(id).get();
335 |     if (!doc.exists) return null;
336 |     return doc.data();
337 | }

```

Archivo: Back/src/services/assignmentWorkflow.service.js (lineas 72-90)

```

72 |     await db().runTransaction(async (tx) => {
73 |         const snap = await tx.get(ref);
74 |         if (!snap.exists) {
75 |             const err = new Error("Not found");
76 |             err.code = 404;
77 |             throw err;
78 |         }
79 |         const a = snap.data();
80 |
81 |         if (a.client_id !== client_id) {
82 |             const err = new Error("Forbidden");
83 |             err.code = 403;
84 |             throw err;
85 |         }
86 |
87 |         if (a.chat_status === "CLOSED") {
88 |             const err = new Error("Chat is closed");
89 |             err.code = 409;
90 |             throw err;

```

2.4 - Validacion en Carga de Archivos

Estado: Cumple

Como demostrarlo: En Admin > Plantillas intentar subir un archivo que no sea .docx. Mostrar validacion de extension, MIME, ZIP y macros.

Archivo: Back/src/controllers/adminContracts.controller.js (lineas 27-38)

```

27 | const upload = multer({
28 |     storage: multer.memoryStorage(),
29 |     limits: { fileSize: MAX_DOCX_MB * 1024 * 1024 },
30 | }).single("file");
31 |
32 | function asyncUpload(req, res) {
33 |     return new Promise((resolve, reject) => {
34 |         upload(req, res, (err) => {
35 |             if (err) return reject(err);
36 |             return resolve();
37 |         });
38 |     });

```

Archivo: Back/src/controllers/adminContracts.controller.js (lineas 192-217)

```

192 | // ===== Upload contrato =====
193 | async function uploadContract(req, res) {
194 |     try {

```

```

195 |     await asyncUpload(req, res);
196 |   } catch (err) {
197 |     if (err && err.code === "LIMIT_FILE_SIZE") {
198 |       return fail(res, `File too large. Max ${MAX_DOCX_MB}MB`, 413);
199 |     }
200 |     return fail(res, "Upload error", 400, { error: String(err.message || err) });
201 |   }
202 |
203 |   const parsed = createContractSchema.safeParse(req.body);
204 |   if (!parsed.success) return fail(res, "Invalid body", 400, parsed.error.flatten());
205 |
206 |   const file = req.file;
207 |   if (!file) return fail(res, "File required (field: file)", 400);
208 |
209 |   try {
210 |     await validateDocxOrThrow({
211 |       buffer: file.buffer,
212 |       originalname: file.originalname,
213 |       mimetype: file.mimetype,
214 |     });
215 |   } catch (e) {
216 |     return fail(res, e.message || "Invalid docx", e.code || 400, e.details);
217 |   }

```

Archivo: Back/src/services/docxValidation.service.js (lineas 3-63)

```

3 | function ensureDocxBasics({ originalname, mimetype }) {
4 |   const name = String(originalname || "").toLowerCase();
5 |   if (!name.endsWith(".docx")) {
6 |     const err = new Error("Only .docx files are allowed");
7 |     err.code = 400;
8 |     throw err;
9 |   }
10 |   if (name.endsWith(".docm")) {
11 |     const err = new Error("Macro-enabled files (.docm) are not allowed");
12 |     err.code = 400;
13 |     throw err;
14 |   }
15 |
16 |   // mimetype típico de docx
17 |   const okMime =
"application/vnd.openxmlformats-officedocument.wordprocessingml.document";
18 |   if (mimetype && mimetype !== okMime) {
19 |     const err = new Error("Invalid file type (mimetype)");
20 |     err.code = 400;
21 |     throw err;
22 |   }
23 | }
24 |
25 | function ensureZipHeader(buffer) {
26 |   // docx es zip, debe iniciar con 'PK'
27 |   if (!Buffer.isBuffer(buffer) || buffer.length < 4) {
28 |     const err = new Error("Invalid file");
29 |     err.code = 400;
30 |     throw err;
31 |   }
32 |   const pk = buffer.slice(0, 2).toString("utf8");
33 |   if (pk !== "PK") {
34 |     const err = new Error("Invalid .docx (not a zip container)");
35 |     err.code = 400;
36 |     throw err;
37 |   }
38 | }
39 |
40 | async function validateDocxOrThrow({ buffer, originalname, mimetype }) {
41 |   ensureDocxBasics({ originalname, mimetype });
42 |   ensureZipHeader(buffer);
43 |
44 |   const zip = await JSZip.loadAsync(buffer);
45 |
46 |   // Debe existir el documento principal
47 |   const hasDocumentXml = Boolean(zip.file("word/document.xml"));
48 |   if (!hasDocumentXml) {
49 |     const err = new Error("Invalid .docx structure (missing word/document.xml)");
50 |     err.code = 400;
51 |     throw err;
52 |   }
53 |
54 |   // Si trae macros -> rechazar
55 |   const hasMacro = Boolean(zip.file("word/vbaProject.bin"));
56 |   if (hasMacro) {
57 |     const err = new Error("Macro content detected; rejected");
58 |     err.code = 400;
59 |     throw err;
60 |   }
61 |
62 |   return true;

```

3.1 - Cifrado de Contraseñas

Estado: Cumple

Como demostrarlo: Mostrar bcrypt.genSalt(12), bcrypt.hash y verifyPassword. No mostrar contraseñas reales.

Archivo: Back/src/utills/password.js (lineas 1-12)

```
1 | const bcrypt = require("bcryptjs");
2 |
3 | async function hashPassword(plain) {
4 |     const salt = await bcrypt.genSalt(12);
5 |     return bcrypt.hash(plain, salt);
6 | }
7 |
8 | async function verifyPassword(plain, hash) {
9 |     return bcrypt.compare(plain, hash);
10 | }
11 |
12 | module.exports = { hashPassword, verifyPassword };
```

Archivo: Back/src/controllers/adminUsers.controller.js (lineas 20-35)

```
20 |     const parsed = createUserSchema.safeParse(req.body);
21 |     if (!parsed.success) return fail(res, "Invalid body", 400, parsed.error.flatten());
22 |
23 |     const { name, email, role, position } = parsed.data;
24 |
25 |     const exists = await usersService.getByEmail(email);
26 |     if (exists) return fail(res, "Email already exists", 409);
27 |
28 |     // Creamos un password aleatorio (NO se entrega). El usuario establecerá el suyo vía
29 |     link seguro.
30 |     const tempPassword = generateTempPassword();
31 |     const password_hash = await hashPassword(tempPassword);
32 |
33 |     const created = await usersService.create({
34 |         name,
35 |         email,
36 |         role,
```

Archivo: Back/src/controllers/passwordReset.controller.js (lineas 66-82)

```
66 |     const parsed = confirmResetSchema.safeParse(req.body);
67 |     if (!parsed.success) return fail(res, "Invalid body", 400, parsed.error.flatten());
68 |
69 |     const { token, newPassword } = parsed.data;
70 |
71 |     const tokenDoc = await passwordTokensService.getValidByRawToken(token);
72 |     if (!tokenDoc) return fail(res, "Token invalid or expired", 400);
73 |
74 |     const userId = tokenDoc.data.user_id;
75 |     const user = await usersService.getById(userId);
76 |     if (!user) return fail(res, "User not found", 404);
77 |     if (user.status !== "ACTIVE") return fail(res, "User disabled", 403);
78 |
79 |     const password_hash = await hashPassword(newPassword);
80 |     const profile_completed = hasCompletedProfile(user);
81 |
82 |     await usersService.patch(userId, {
```

3.2 - Principio de Menor Privilegio (RBAC)

Estado: Cumple

Como demostrarlo: Entrar como admin y cliente. En código mostrar authJWT, requireRole y rutas ADMIN/CLIENT.

Archivo: Back/src/middlewares/authJWT.js (lineas 3-18)

```
3 | function authJWT(req, res, next) {
4 |     const header = req.headers.authorization || "";
5 |     const [type, token] = header.split(" ");
6 |
7 |     if (type !== "Bearer" || !token) {
8 |         return res.status(401).json({ ok: false, message: "Missing Bearer token" });
9 |     }
10 |
11 |     const decoded = jwt.decode(token, { complete: false });
12 |     if (!decoded) return res.status(401).json({ ok: false, message: "Invalid token" });
13 |
14 |     const { role } = decoded.payload;
15 |     if (!role) return res.status(401).json({ ok: false, message: "Invalid token" });
16 |
17 |     const { requireRole } = req;
18 |     if (requireRole !== role) return res.status(403).json({ ok: false, message: "Forbidden" });
19 |
20 |     next();
21 | }
```



```

9 |     }
10 |
11 |     try {
12 |         const payload = verifyToken(token);
13 |         req.user = payload; // { userId, role }
14 |         return next();
15 |     } catch {
16 |         return res.status(401).json({ ok: false, message: "Invalid or expired token" });
17 |     }
18 | }

```

Archivo: Back/src/middlewares/requireRole.js (lineas 1-13)

```

1 | function requireRole(...roles) {
2 |     return (req, res, next) => {
3 |         if (!req.user?.role) {
4 |             return res.status(401).json({ ok: false, message: "Unauthorized" });
5 |         }
6 |         if (!roles.includes(req.user.role)) {
7 |             return res.status(403).json({ ok: false, message: "Forbidden" });
8 |         }
9 |         return next();
10 |     };
11 | }
12 |
13 | module.exports = requireRole;

```

Archivo: Back/src/routes/admin.routes.js (lineas 1-13)

```

1 | const router = require("express").Router();
2 | const authJWT = require("../middlewares/authJWT");
3 | const requireRole = require("../middlewares/requireRole");
4 | const ctrl = require("../controllers/adminUsers.controller");
5 |
6 | // Solo ADMIN puede entrar
7 | router.use(authJWT, requireRole("ADMIN"));
8 |
9 | router.post("/users", ctrl.createUser);
10 | router.get("/users", ctrl.listUsers);
11 | router.get("/users/:id", ctrl.getUser);
12 | router.patch("/users/:id", ctrl.patchUser);
13 | router.patch("/users/:id/disable", ctrl.disableUser);

```

Archivo: Back/src/routes/clientAssignments.routes.js (lineas 1-12)

```

1 | const router = require("express").Router();
2 | const authJWT = require("../middlewares/authJWT");
3 | const requireRole = require("../middlewares/requireRole");
4 | const c = require("../controllers/clientAssignments.controller");
5 | const profile = require("../controllers/clientProfile.controller");
6 |
7 | // Protegido: CLIENT
8 | router.use(authJWT, requireRole("CLIENT"));
9 |
10 | router.get("/profile", profile.getProfile);
11 | router.patch("/profile", profile.updateProfile);
12 |

```

3.3 - Seguridad de Tokens (JWT)

Estado: Cumple

Como demostrarlo: Hacer login y mostrar DevTools > Local Storage > seal_token. En código mostrar payload mínimo y expiración.

Archivo: Back/src/utills/token.js (lineas 1-12)

```

1 | const jwt = require("jsonwebtoken");
2 | const { JWT_SECRET, JWT_EXPIRES_IN } = require("../config/env");
3 |
4 | function signToken(payload) {
5 |     return jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
6 | }
7 |
8 | function verifyToken(token) {
9 |     return jwt.verify(token, JWT_SECRET);
10 | }
11 |
12 | module.exports = { signToken, verifyToken };

```

Archivo: Back/src/controllers/auth.controller.js (lineas 20-39)

```

20 |     await userService.patch(user.id, { last_login_at: new Date().toISOString() });
21 |

```

```

22 |     const token = signToken({ userId: user.id, role: user.role });
23 |
24 |     const profile_completed = Boolean(user.profile_completed);
25 |
26 |     return ok(res, {
27 |         token,
28 |         user: {
29 |             id: user.id,
30 |             name: user.name,
31 |             email: user.email,
32 |             role: user.role,
33 |             must_change_password: user.must_change_password,
34 |             status: user.status,
35 |
36 |             profile_completed,
37 |         },
38 |         needs_profile_completion: !profile_completed,
39 |     });

```

Archivo: Front/src/lib/api.ts (lineas 339-368)

```

339 | export async function apiFetch<T>(path: string, options: ApiFetchOptions = {}) {
340 |     const { auth = true, headers, ...init } = options;
341 |     const finalHeaders = new Headers(headers);
342 |
343 |     const isFormData = typeof FormData !== "undefined" && init.body instanceof FormData;
344 |
345 |     if (!isFormData && init.body !== undefined && !finalHeaders.has("Content-Type")) {
346 |         finalHeaders.set("Content-Type", "application/json");
347 |     }
348 |
349 |     if (auth) {
350 |         const token = getToken();
351 |         if (token) finalHeaders.set("Authorization", `Bearer ${token}`);
352 |     }
353 |
354 |     const res = await fetch(buildUrl(path), {
355 |         ...init,
356 |         headers: finalHeaders,
357 |     });
358 |
359 |     const payload = await readPayload<T>(res);
360 |
361 |     if (!res.ok || !payload?.ok) {
362 |         if (res.status === 401) logout();
363 |
364 |         throw new ApiError(
365 |             payload?.message || payload?.error || `Error ${res.status}`,
366 |             res.status,
367 |             payload?.details
368 |         );

```

3.4 - Atributos de Cookies Seguras

Estado: No cumple

Como demostrarlo: Mostrar en DevTools que la sesion vive en Local Storage, no en cookies
HttpOnly/Secure/SameSite.

Archivo: Front/src/lib/auth.ts (lineas 13-24)

```

13 | const TOKEN_KEY = "seal_token";
14 | const USER_KEY = "seal_user";
15 |
16 | function canUseStorage() {
17 |     return typeof window !== "undefined" && typeof window.localStorage !== "undefined";
18 | }
19 |
20 | export function saveSession(token: string, user: SessionUser) {
21 |     if (!canUseStorage()) return;
22 |     localStorage.setItem(TOKEN_KEY, token);
23 |     localStorage.setItem(USER_KEY, JSON.stringify(user));
24 | }

```

Archivo: Front/src/lib/auth.ts (lineas 33-59)

```

33 | export function getToken() {
34 |     if (!canUseStorage()) return null;
35 |     return localStorage.getItem(TOKEN_KEY);
36 | }
37 |
38 | export function getUser() {
39 |     if (!canUseStorage()) return null;

```

```

40 |   const user = localStorage.getItem(USER_KEY);
41 |   if (!user) return null;
42 |
43 |   try {
44 |     return JSON.parse(user) as SessionUser;
45 |   } catch {
46 |     logout();
47 |     return null;
48 |   }
49 | }
50 |
51 | export function hasSession() {
52 |   return Boolean(getToken() && getUser());
53 | }
54 |
55 | export function logout() {
56 |   if (!canUseStorage()) return;
57 |   localStorage.removeItem(TOKEN_KEY);
58 |   localStorage.removeItem(USER_KEY);
59 | }

```

Archivo: Front/src/app/(dashboard)/layout.tsx (lineas 17-24)

```

17 | function getSessionSnapshot() {
18 |   if (typeof window === "undefined") return "";
19 |   return `${window.localStorage.getItem(TOKEN_KEY) || ""}\n${window.localStorage.getItem(USER_KEY) || ""}`;
20 | }
21 |
22 | function getServerSessionSnapshot() {
23 |   return "";
24 | }

```

4.1 - Manejo Correcto de CORS

Estado: Cumple parcialmente

Como demostrarlo: Mostrar Back/src/app.js: usa cors(), pero sin origen restringido. Explicar que en produccion debe limitarse.

Archivo: Back/src/app.js (lineas 1-5)

```

1 | const express = require("express");
2 | const helmet = require("helmet");
3 | const cors = require("cors");
4 | const morgan = require("morgan");
5 | const rateLimit = require("express-rate-limit");

```

Archivo: Back/src/app.js (lineas 33-37)

```

33 | app.use(helmet());
34 | app.use(cors());
35 | app.use(express.json({ limit: "1mb" }));
36 | app.use(morgan("dev"));
37 | app.use(apiLimiter);

```

4.2 - Helmet / Encabezados HTTP

Estado: Cumple

Como demostrarlo: Con backend prendido ejecutar curl -I http://localhost:3001/health y mostrar headers de seguridad.

Archivo: Back/src/app.js (lineas 1-5)

```

1 | const express = require("express");
2 | const helmet = require("helmet");
3 | const cors = require("cors");
4 | const morgan = require("morgan");
5 | const rateLimit = require("express-rate-limit");

```

Archivo: Back/src/app.js (lineas 22-39)

```

22 | const apiLimiter = rateLimit({
23 |   windowMs: 60 * 1000,
24 |   limit: 50,
25 |   standardHeaders: true,
26 |   legacyHeaders: false,
27 |   message: {

```

```

28 |         ok: false,
29 |         message: "Demasiadas peticiones desde esta IP. Intenta de nuevo en un minuto.",
30 |     },
31 | });
32 |
33 | app.use(helmet());
34 | app.use(cors());
35 | app.use(express.json({ limit: "1mb" }));
36 | app.use(morgan("dev"));
37 | app.use(apiLimiter);
38 |
39 | app.get("/health", (req, res) => res.json({ ok: true }));

```

5.1 - Abstraccion de Errores en Produccion

Estado: Cumple

Como demostrarlo: Ejecutar `curl -i http://localhost:3001/admin/users` sin token. Debe devolver 401 sin stack trace.

Archivo: Back/src/middlewares/errorHandler.js (lineas 1-4)

```

1 | module.exports = function errorHandler(err, req, res, next) {
2 |     console.error("■", err);
3 |     return res.status(500).json({ ok: false, message: "Internal server error" });
4 | };

```

Archivo: Back/src/utills/response.js (lineas 1-18)

```

1 | function isValidHttpStatus(code) {
2 |     const n = Number(code);
3 |     return Number.isInteger(n) && n >= 100 && n <= 999;
4 | }
5 |
6 | function ok(res, data, status = 200) {
7 |     const s = isValidHttpStatus(status) ? status : 200;
8 |     return res.status(s).json({ ok: true, data });
9 | }
10 |
11 | function fail(res, message, status = 400, details) {
12 |     const s = isValidHttpStatus(status) ? status : 500;
13 |     const payload = { ok: false, message: message || "Error" };
14 |     if (details !== undefined) payload.details = details;
15 |     return res.status(s).json(payload);
16 | }
17 |
18 | module.exports = { ok, fail };

```

Archivo: Back/src/middlewares/authJWT.js (lineas 7-17)

```

7 |     if (type !== "Bearer" || !token) {
8 |         return res.status(401).json({ ok: false, message: "Missing Bearer token" });
9 |     }
10 |
11 |     try {
12 |         const payload = verifyToken(token);
13 |         req.user = payload; // { userId, role }
14 |         return next();
15 |     } catch {
16 |         return res.status(401).json({ ok: false, message: "Invalid or expired token" });
17 |     }

```

5.2 - Logs de Seguridad Centralizados

Estado: Cumple parcialmente

Como demostrarlo: Dejar visible la terminal del backend: Morgan registra requests. Explicar que falta logging centralizado tipo Winston/Pino/Sentry.

Archivo: Back/src/app.js (lineas 1-6)

```

1 | const express = require("express");
2 | const helmet = require("helmet");
3 | const cors = require("cors");
4 | const morgan = require("morgan");
5 | const rateLimit = require("express-rate-limit");
6 |

```

Archivo: Back/src/app.js (lineas 33-37)

```

33 | app.use(helmet());
34 | app.use(cors());
35 | app.use(express.json({ limit: "1mb" }));
36 | app.use(morgan("dev"));
37 | app.use(apiLimiter);

```

Archivo: Back/src/services/gemini.service.js (lineas 133-139)

```

133 |     if (!r.ok) {
134 |         const t = await r.text();
135 |         console.error("[GEMINI] HTTP", r.status, t);
136 |         const err = new Error(`Gemini API error (${r.status})`);
137 |         err.code = 502;
138 |         err.details = { status: r.status, body: t };
139 |         throw err;

```

Archivo: Back/src/jobs/draftCleanup.job.js (lineas 15-27)

```

15 |         try {
16 |             const result = await contractDraftsService.cleanupExpiredDrafts();
17 |             console.log(`[draftCleanup] cleaned=${result.cleaned}
| scanned=${result.scanned}`);
18 |         } catch (err) {
19 |             console.error("[draftCleanup] error:", err);
20 |         }
21 |     },
22 |     { timezone: APP_TZ }
23 | );
24 |
25 | task.start();
26 | console.log(`[draftCleanup] scheduled cron="${DRAFT_CLEANUP_CRON}" tz="${APP_TZ}"`);
27 | return task;

```